物品管理 App 创意方案集

六个方向 · 功能拆解 · 实施排期 · 验收自检

对应题目：题目一 物品管理 App（原生客户端）

配套文档：《SwiftUI 界面设计自学手册》

选定方向之后，界面设计请配合《SwiftUI 界面设计自学手册》的任务卡一起使用

目 录

第 0 章 先看结论：怎么选一个你能做完的方案 4

0.1 这份方案集解决什么问题 4

0.2 选题的三个硬标准 4

0.3 六个方向一览 5

0.4 我的推荐 5

0.5 怎么用这份方案集 6

第 1 章 把题目要求翻译成功能清单 7

1.1 基础要求逐条拆解 7

1.2 进阶要求：每个点给你几种做法 8

1.3 必须做与加分项的边界 10

1.4 功能总表 11

第 2 章 六个创意方向 13

2.1 怎么读这一章 13

2.2 方向一 保质期管家 13

2.3 方向二 数码抽屉与保修簿 14

2.4 方向三 收纳定位：东西放在哪 15

2.5 方向四 我的收藏图鉴 16

2.6 方向五 礼物与人情账本 18

2.7 方向六 借还与共享 19

2.8 六个方向横向对比 20

第 3 章 推荐方案：保质期管家 21

3.1 为什么推荐这个方向 21

3.2 MVP 功能地图 21

3.3 三个可选的亮点模块 22

3.4 界面地图 23

3.5 数据模型与字段 24

3.6 技术选型清单 25

3.7 这个方案能拿到什么评价.....25

第 4 章 让作品显得有创意的 18 个低成本点子.....27

4.1 交互细节类.....27

4.2 视觉呈现类.....28

4.3 原生能力类.....28

4.4 工程与演示类.....29

第 5 章 14 天实施排期.....31

5.1 三个里程碑.....31

5.2 逐日计划.....31

5.3 每天收工前的自检.....32

5.4 时间只剩一半怎么办.....33

第 6 章 这个题目上最容易翻车的 12 个地方.....34

6.1 数据与图片.....34

6.2 交互与状态.....35

6.3 时间与提醒.....37

6.4 演示与提交.....37

第 7 章 对照题目要求的验收自检表.....38

7.1 基础要求自检.....38

7.2 进阶要求自检.....38

7.3 代码与工程自检.....39

7.4 三分钟演示脚本.....39

7.5 提交前最后一遍.....40

第 0 章 先看结论：怎么选一个你能做完的方案

0.1 这份方案集解决什么问题

题目原文只有一段话加九条要求，看起来很自由，实际上埋着一个新手最容易掉的坑：**自由选题最大的风险不是想不出点子，而是选了一个做不完的点子。**

现实约束很清楚：你大概有一到两周、每天两三个小时，要同时完成建模、界面、图片、搜索、调试和演示。在这个约束下，"想法够不够炫"远远不如"能不能按期交付"重要。一份功能完整、能演示、没有明显破绽的作品，评分一定高于一份功能宏大但处处半成品、点两下就崩的作品。

所以这份方案集不直接替你决定做什么，而是给你三样东西：

- 把题目的九条要求翻译成可执行的功能清单和验收标准，让你知道及格线画在哪里。
- 六个风格不同的创意方向，每个都写清楚用户场景、创意亮点、工作量、风险点，你可以按自己的兴趣和实力挑。
- 一个推荐方案的完整落地路径：功能地图、界面地图、数据模型、技术选型、14 天排期、避坑清单、验收自检表。

界面具体长什么样、怎么排布、用什么配色，那份工作交给配套的《SwiftUI 界面设计自学手册》，你可以自己独立完成。

0.2 选题的三个硬标准

挑方向的时候，只问三个问题。

第一，做得完吗。 把方向里所有功能按"每个功能大约半天"估一遍，如果总数超过 20 个半天，就要砍。特别注意那些需要额外权限或额外框架的功能：通知、相机、定位、小组件、云同步，每一个都会额外吃掉半天到一天，不是加一行代码就能用的。

第二，有故事吗。 演示的时候你要能用一句话说清"这个 App 是给谁解决什么问题的"。有故事的方向天然好讲：给租房学生管食材保质期、给爱折腾数码的人管保修、给搬过家的人找东西。反过来，一个什么都管的"万能物品管理器"没有故事，演示时只能说"我可以添加物品、我可以删除物品"，听起来就像课程作业。

第三，有加分点吗。 题目给了四条进阶要求，你至少要有一条能做出说得出口的效果。做得深比铺得广好：一个真正能用的到期提醒，比四个都只做了一半的进阶功能更容易拿到好评价。

> 一个可以直接用的判断方法：把你想做的方向说给室友听。如果他能立刻说出"这个我确实需要"，方向就对了；如果他说"这不就是个记事本吗"，换个方向，或者把亮点做深。

0.3 六个方向一览

方向	一句话	创意度	工作量
保质期管家	管食物药品化妆品的到期时间	中	小到中
数码抽屉与保修簿	管电子产品、线材、保修期	中	中
收纳定位	记录东西放在哪个房间哪个箱子	高	中
我的收藏图鉴	书籍、唱片、手办的封面墙与统计	高	中到大
礼物与人情账本	记录送出去的礼物和人情往来	高	中
借还与共享	记录借给谁、什么时候还	中高	中

六个方向的详细分析在第 2 章，包括每个方向的用户场景、创意亮点、核心界面、数据模型、覆盖的进阶要求、工作量评估、风险点，以及"适合什么样的你"。

0.4 我的推荐

如果让我替你选，我推荐**方向一「保质期管家」**，理由有三条。

它的基础要求是天然成立的。 物品就是食品、药品、化妆品；分类天然存在（生鲜、零食、药品、护肤）；图片有明确用途（拍下包装或实物）；到期日期给了排序和筛选一个真实的目的。你不需要硬凑分类，也不需要为了"支持图片"而随便加个图片字段。

它的进阶要求最容易做出效果。 题目的第一条进阶要求是"物品到期、保修或其他时间提醒"，而这正好是这个方向的核心。你可以从最简单的"列表上显示剩余天数并标色"起步，再到"按临期排序"，最后做到"本地通知到点提醒"，每一步都能独立交付、独立演示。

它的演示效果最好。 三分钟的演示里，你可以讲出一个完整的故事：拍一瓶牛奶的包装进 App，它自动出现在"临期"分组里标成橙色，告诉你还有 3 天到期，后天它给你发一条通知，你喝完把它删掉。这个故事有起点、有冲突、有结局，比"我能增删改查"强得多。

推荐方案的完整落地路径在第 3 章，包括功能地图、界面地图、数据模型、技术选型，以及三个成本可控的亮点模块（到期提醒、存放位置、分享与小组件）。第 5 章给出 14 天的逐日排期。

需要说清的局限：这个方向的题目竞争可能比较激烈，容易撞车，所以**你和别人的差距会体现在细节上**——临期阈值的分级、空状态的处理、统计页的可读性、深色模式的一致性。第 4 章列的 18 个低成本点子就是用来拉开这个差距的。

0.5 怎么用这份方案集

按你的状态选一条路径。

还没决定做什么：读第 1 章了解及格线，再读第 2 章的六个方向，挑出最让你有话说的一到两个，然后看第 3 章推荐的方案能不能说服你。

已经决定做保质期管家：跳过第 2 章，直接按第 3 章的功能地图和第 5 章的排期开工。每做完一天，用第 7 章的自检表对一遍。

已经决定做别的方向：第 1 章、第 4 章、第 5 章的排期思路、第 6 章、第 7 章都通用，只有第 3 章是专门写给保质期管家的。你可以照第 3 章的结构（功能地图、界面地图、数据模型、技术选型）给自己的方向写一份。

准备开始写界面：打开配套的《SwiftUI 界面设计自学手册》。第 1 章给你设计规范，第 2 章给你从需求到界面的流程，第 3 章是布局查字典的地方，第 5 章的十张任务卡直接对应你要做的十个界面，第 8 章的挑战是留给你自己发挥的部分。

> 这份方案集里所有的代码和字段名都遵循同一套约定，可以在项目的任何位置直接使用。数据模型的完整定义见配套自学手册第 4 章。

第 1 章 把题目要求翻译成功能清单

题目只有五行字，但评委和答辩老师看的不是你写了多少行代码，而是这五条能不能一条一条对上。这一章把题目原文拆成可以直接动手的功能点，每条都给出验收标准和"偷懒做法为什么会被看出来"，然后展开四条进阶要求，最后给出一张可以照着排期的总表。

1.1 基础要求逐条拆解

第一条：添加、查看、编辑、删除。 这一条要的是一条闭环的数据流，不是四个孤立按钮。要做什么：列表页能点进详情页（查看），列表页有加号或工具栏按钮进入新增表单，详情页或列表左滑能进入编辑，编辑页和新增页共用同一套表单视图，列表左滑和详情页各有一个删除入口。验收标准：从列表能进入表单；在表单里能选图片；填完点保存后新物品立刻出现在列表顶部，不需要手动下拉刷新；点进详情能看到刚才填的全部字段；在详情页改掉名称后返回，列表里那一行同时变了；删除后列表少一行，重启 App 不会复活；杀进程重开，列表内容和图片都还在。常见的偷懒做法有四种。只做新增和列表，编辑按钮点了没反应或者只弹一个提示。详情页和编辑页拆成两套表单，两边字段不一致，改一边漏一边。删除只有一个入口，而且没有二次确认，答辩时误删一条演示数据。用内存数组装数据，重启就空，只能靠反复操作掩盖。这些做法有个共同特点：单点操作看起来正常，一连起来就断。

第二条：至少包含名称、分类，并支持图片。 这一条要的字段是"真实落库"，不是界面上摆样子。要做什么：名称、分类必填，数量、存放位置、备注、到期日选填；图片能拍照也能从相册选，能替换、能删除；列表行显示小缩略图，详情页显示大图。验收标准：保存一条只有名称的物品能成功，保存一条什么都没填的会被拦住并给出提示；选图后列表缩略图立刻出现；一百条带图物品的列表滚动不卡顿。技术要点：图片用 `@Attribute(.externalStorage)` 存到数据库外部文件，不要直接把大图塞进普通属性；选图用 `PhotosPicker`，权限由系统选择器处理；存之前把图压到长边 1600 像素以内，按 JPEG 质量 0.8 导出。常见的偷懒做法：所有字段都叫"备注"，界面上摆一个图片框但从不赋值；把原图 `Data` 直接塞进普通属性，二十条之后列表掉帧；用户拒绝相册权限时直接黑屏，不写任何说明。

第三条：本地保存，重开还能看。 这一条要的是持久化，不是"这次演示没崩"。要做什么：用 SwiftData 的 `ModelContainer` 在 App 入口挂上容器，列表用 `@Query` 读数据，增删改都走 `modelContext`。验收标准：添加三条物品，把 App 从多任务列表上划掉，重新打开，三条都在；编辑一条后同样操作，改动还在；带图的物品重开后图片仍然显示；关掉网络或开飞行模式，全部功能照常可用。常见的偷懒做法：数据只放在视图的状态变量里；把数据写成沙盒里的 JSON 文件但图片存在缓存目录，系统清理后图全没了；只在模拟器上测过没杀过进程，答辩现场一划掉就露馅。这是五条里唯一"没法在界面上伪装"的一条，必须最早做完。

第四条：分类、搜索或筛选。 这一条只要满足一个就及格，但满足一个和做好差距很大。最少要做的：分类筛选加一个搜索框。验收标准：搜索能匹配名称、备注、存放位置和标签，输入两个字列表立刻减少，清空后完整恢复；分类筛选用一条横向滚动的分类标签，点击后只显示该类；搜索和分类可以同时生效，即先筛分类再搜关键词，结果是交集不是并集；筛完没有结果时显示明确的空状态，而不是一片空白。技术要点：搜索用 `searchable` 修饰符，可以叠加在 `NavigationStack` 上；排序默认按 `createdAt` 倒序，不要按名称排，中文字符串排序的结果不符合直觉。常见的偷懒做法：搜索只在按回车后才触发，或者只能匹配名称的完整前缀；分类筛选和搜索互相覆盖，选完分类搜索框就失效；筛选结果为空时列表区域什么都不显示，用户以为是卡了。

第五条：完整、可实际使用的流程。 这一条是前四条的合并验收，也是最容易失分的一条。要做什么：App 里不存在"死路"页面，每个屏幕都能返回，每个操作都有可见反馈。验收标准：首次启动能看到空状态和一句"点右上角加号添加第一件物品"的引导；连续添加三条，列表每次都立刻更新；把手机交给一个没用过的人，三分钟内他能独立完成添加、查看、编辑、删除和搜索；每个动作都有反馈，保存有提示或直接刷新列表，删除有确认，失败有错误说明；全程离线可用。常见的偷懒做法：只有新建没有编辑和删除，导航栏的编辑按钮是装饰；删除没有二次确认，误删后没有撤销；错误信息只写在控制台，界面上毫无提示；空状态是一张白屏，第一次打开的人不知道该干什么。

1.2 进阶要求：每个点给你几种做法

第一条：时间提醒。 时间提醒的完整形态是一条链路：数据里有一个日期，界面上能看到还剩多久，列表能按紧急程度排序，到点能主动通知，最后能在主屏上一眼看到。你可以在这条链路的任意一段停下，四种做法的难度差别很大。

做法	难度	所需技术	做完能看到什么效果
详情页显示剩余天数	★☆☆	`Date` 与 `Calendar` 的日期差、计算属性	详情页出现"还剩 3 天""已过期 12 天"，按剩余天数换颜色
列表按临期排序并标色	★☆☆	`@Query` 排序、`List` 行修饰符	切到"临期"分组，最紧急的排最前，行首一条橙色或红色色条
本地通知到点提醒	★★☆	`UserNotifications`、通知授权、`UNCalendarNotificationTrigger`	到点当天早上手机弹出提醒，通知中心可查历史
主屏 Widget 显示临期数量	★★★	`WidgetKit`、`App Group`、共享数据读取	主屏小组件显示"临期 3 件"，点一下跳进 App

推荐路径是先做前两种，它们只用到日期差计算和列表排序，半天能见效；通知要处理授权被拒绝的分支，至少留半天；Widget 涉及新建 Target 和数据共享，独立占一天。

第二条：更丰富的分类、标签、排序和统计。 这一条是"数据看得见"的一组能力，四档做法如下。

做法	难度	所需技术	做完能看到什么效果
多标签与标签筛选	★☆☆	`tags` 数组字段、`Set` 的包含判断	一条物品挂多个标签，筛选栏点标签后列表只剩带该标签的
看板统计	★★☆	`Charts` 框架、按属性分组聚合	首页一块统计区，分类占比环形图加每月新增柱状图
可排序表格	★★☆	`Table` 或自定义行、`Picker` 切换排序键	表格式浏览，点表头按数量、到期日、创建时间排序
独立分类模型	★★★	第二个 `@Model`、关系、数据迁移	分类可自定义图标和颜色，能进分类管理页增删改

统计看板是性价比最高的一项，`Charts` 的柱状图和扇形图各不到二十行，答辩时视觉冲击也强。独立分类模型要注意迁移成本：`categoryName` 从字符串换成关系以后，旧数据要么写迁移代码，要么接受开发期清空重来。

第三条：相机、系统分享和 Widget。 这一条考的是"你和系统打交道的经验"，四档做法如下。

做法	难度	所需技术	做完能看到什么效果
从相册选图	★☆☆	`PhotosPicker`、`Transferable`	表单里点一下弹出系统相册，选完缩略图立刻显示
调用相机拍照	★★☆	`AVCaptureDevice` 权限、`UIImagePickerController` 的 SwiftUI 封装	表单里多一个"拍照"按钮，拍完直接成为物品图片

做法	难度	所需技术	做完能看到什么效果
系统分享	★★☆	`ShareLink`、`Transferable` 或 `UIActivityViewController`	详情页分享按钮，弹出系统分享面板，能把物品信息发到微信或备忘录
主屏 Widget	★★★	`WidgetKit`、`App Group`、时间线刷新	主屏小组件列出三件临期物品，点击跳转详情

前两项解决"物品怎么录进去"的真实痛点，评委很吃这一套。分享功能代码量极小，半小时能加完，但演示效果好，几乎不会失败。Widget 单独放最后，因为它需要新建 Target、配置 App Group，是这一组里唯一可能卡住半天以上的。

第四条：大量物品、图片存储和数据管理优化。 这一条没有明确的功能边界，做的是"工程成熟度"，四档做法如下。

做法	难度	所需技术	做完能看到什么效果
缩略图与内存缓存	★★☆	`ImageIO` 降采样、`NSCache`	一次导入一百条带图数据，列表滚动依然顺滑不闪烁
分批加载	★☆☆	`@Query` 的 `fetchLimit`、分页触发	数据上千条时首屏打开时间明显变短，滚动到底部再加载
导出与导入	★★☆	自定义 `Codable` 副本、`ShareLink`	能把全部数据导出成一个文件分享出去，也能从文件恢复
iCloud 同步与备份	★★★	`CloudKit` 容器、`ModelConfiguration` 的云配置	两台设备登录同一账号，A 添加 B 能看到

优化的前提是先把数据造出来：自己写一个生成随机测试数据的入口，能在十秒内加两百条物品。导出功能很适合答辩，现场演示"导出备份再清库再恢复"，一句"数据不会丢"比十句介绍有用。iCloud 需要付费开发者账号和真机，没有账号就换成"导出到文件"，效果相近。

1.3 必须做与加分项的边界

先把结论说清楚：五条基础要求是及格线，进阶要求是加分项，两者的关系不是"能做多少做多少"，而是"越过了线再加分"。原因是评分表里基础分是刚性分，缺一条就扣一条；进阶分是弹性分，做四个半成品得到的分数远低于做两个完整的。

具体的执行原则有三条。第一，基础要求全部做完之前，不要碰进阶要求，哪怕进阶里最简单的"详情页显示剩余天数"也不要做，因为它会占用你改基础 bug 的时间。第二，判断"做完"的标准是上一

节的验收标准，不是"代码写完了"，能杀进程重开、能交给别人用、能离线跑通，才算做完。第三，进阶最多选两个，选的时候优先挑能串起主线的，比如临期排序和本地通知是一组，统计看板和多标签是一组，单独做 Widget 而不做临期列表就会很突兀。

如果只剩三天，取舍建议是这样。第一天只干一件事：把增删改查、SwiftData 持久化、搜索加分类筛选做通，这一天不做任何美化，界面丑没关系，重点是数据能存能取。第二天做两件性价比最高的事：把详情页的剩余天数加临期配色做了，再把统计看板加在首页，这两项一共不到半天，剩下半天修第一天留下的 bug 并补齐空状态、删除确认、图片压缩这些细节。第三天上午做本地通知，包括申请权限被拒绝时显示一句"去设置里打开通知"，下午只做答辩准备：准备五条演示数据，把"添加带图物品、搜索、改到期日、删除、杀进程重开"这五步排练三遍，确认每一步的反馈都看得见。这三天里不要开新功能，也不要为了好看大改界面样式，改样式是最容易引入新 bug 的操作。

1.4 功能总表

下表把前面提到的功能点全部列出来，最后的阶段栏按七天排期给出建议的动手顺序：D1 单日能完成，D1-D2 两天内完成，D2-D3 排在基础之后，D3-D5 属于进阶，D5-D7 是加分或优化。

功能	对应要求	难度	建议什么时候做
新建物品表单	基础 1	★☆☆	D1
列表页与详情页	基础 1	★☆☆	D1
编辑已有物品	基础 1	★☆☆	D1
左滑与详情页删除	基础 1	★☆☆	D1
名称、分类、数量、位置、备注字段	基础 2	★☆☆	D1
相册选图与缩略图	基础 2	★☆☆	D1-D2
图片压缩与外部存储	基础 2	★★☆	D2
SwiftData 容器与查询	基础 3	★☆☆	D1
搜索名称与备注	基础 4	★☆☆	D2
分类筛选标签条	基础 4	★☆☆	D2
排序切换	基础 4	★☆☆	D2
空状态与错误提示	基础 5	★☆☆	D2
离线可用与全流程自测	基础 5	★☆☆	D2-D3
详情页剩余天数	进阶 1	★☆☆	D3
列表临期排序与配色	进阶 1	★☆☆	D3

功能	对应要求	难度	建议什么时候做
本地通知提醒	进阶 1	★★☆	D3
主屏 Widget 显示临期	进阶 1、3	★★★	D5-D7
多标签与标签筛选	进阶 2	★☆☆	D3
统计看板与图表	进阶 2	★★☆	D3
可排序表格视图	进阶 2	★★☆	D4
独立分类模型与迁移	进阶 2	★★★	D5-D7
相机拍照入图	进阶 2、3	★★☆	D4
系统分享导出	进阶 3	★★☆	D3
缩略图缓存与分批加载	进阶 4	★★☆	D4
数据导出与导入	进阶 4	★★☆	D4-D5
iCloud 同步备份	进阶 4	★★★	D5-D7

第 2 章 六个创意方向

同一个题目，做出来可以是六种完全不同的 App。这一章给出六个方向，它们共用同一套数据模型，区别在于把哪一部分做深、让用户产生什么感觉。选题判断比写代码更早决定成败，所以这一章不贴实现代码，只讲清楚每个方向凭什么不平庸、要花多少时间、会卡在哪。

2.1 怎么读这一章

每个方向都按同一组标签展开，方便你横向对比同一条信息。你不需要读完全部六个方向，正确的用法是先看"一句话定位"和"适合什么样的你"两段，快速排掉三个不心动的，剩下的三个再仔细读核心界面和风险点。判断某个方向是否适合你，问自己三个问题：这个场景我自己有没有真实的使用需求？它的核心界面我能不能在两天内画出来？它最大的风险我有没有办法绕开？三个都答"能"，就选它。

人天按"有经验的 iOS 开发者每天六小时"折算，你们是第一次做，所以下文同时按每天三小时算一遍日历天数。

2.2 方向一 保质期管家

一句话定位：把家里的食品、药品和日用品按到期日管起来，打开 App 第一眼就知道什么快过期了。

用户与场景：适合囤货的人、自己开火的合租房学生，以及实验室里管试剂耗材的同学。典型场景是站在冰箱前，用三十秒把刚买的七件东西录进去，之后每次打开只关心"这周要处理什么"。

创意亮点：它把物品管理从"库存台账"变成"时间轴"。同类作品多在分类列表加搜索上就结束了，你可以在数据层加一个不落库的计算属性 `expiryState`，把物品按剩余天数分成过期、30 天内、安全三档，列表里用左侧色条和倒计时文字表达紧急程度，不点进详情就知道先处理谁。第二个亮点是快捷操作：详情页有"已丢弃"和"已用完"按钮，点击后物品移出主列表并进入历史，列表只显示还值得关心的东西。第三个亮点是"记录日期只花一秒"的表单，数量用步进器、日期默认今天到一年后，让录入速度成为真正的竞争力。

核心界面：主列表用卡片行，每行是缩略图、名称、分类、剩余天数色条；顶部筛选条含全部、临期、已过期、已用完四段；详情页显示大图、倒计时天数和两个状态按钮；新增表单从列表右上角进入。

数据模型要点：核心是 `expireDate`，模型里的 `expiryState` 计算属性就是为它准备的。其次用 `name`、`categoryName`、`quantity`、`location`、`note`、`imageData`、`isFavorite`，其中 `location` 记"冰箱上层"这类位置。建议额外加 `purchaseDate`、`openedDate`、`isConsumed` 三个可选字段，前两个支持"开封后 7 天内用完"的规则，后一个支持历史记录；排序统一用 `expireDate` 升序。

覆盖的进阶要求：进阶一基本全包（剩余天数、临期排序、本地通知、Widget 显示临期数量），进阶二顺带用上标签和统计图表，进阶三可以只加相机拍照。这是对进阶要求覆盖最完整的一个方向。

工作量评估：三到四人天，按每天三小时算大约 8 到 11 天。拆开是数据模型和列表一天、详情与状态操作半天、本地通知半天到一天、统计和打磨半天。如果只做剩余天数和临期排序，两天就能出一个能演示的版本。

风险点：用户懒得填到期日，做出来的临期列表是空的，答辩现场没数据可看；通知权限被拒绝后如果没有降级提示，功能看起来就是失效了；日期比较要处理跨零点，用 `Calendar` 的日期差而不是相减秒数，否则"还剩 0 天"和"已过期"会打架。

适合什么样的你：想稳稳做出一个完整作品、愿意花时间把细节做顺的人。这个方向最容易在一周内做到"真的能用"，缺点是新鲜感一般，需要靠通知和 Widget 把完成度拉起来。

2.3 方向二 数码抽屉与保修簿

一句话定位：给手机、耳机、键盘、充电器和各种配件建一份保修档案，随时知道还在不在保、什么时候买的、多少钱。

用户与场景：适合数码爱好者、有多个设备的同学，以及需要管理实验室仪器的同学。典型场景是设备出问题时，打开 App 就能看到购买日期、保修截止日和购买渠道，而不是翻邮箱和购物记录。

创意亮点：这个方向的差异化不在界面而在信息结构。它把"日期"分成两个语义完全不同的字段：`expireDate` 表示保修到期的上限，`purchaseDate` 表示购买起点，界面上一律显示成"保修剩余 87

天"这种第一人称表述。第二个亮点是"设备护理时间线"，把充电、换膜、清灰、送修这类事件按时间排成一列，`note` 加 `updatedAt` 就够用。第三个亮点是"该扔了"的判断：一台设备保修已过、购入超过三年、又标了 `isFavorite` 为真，界面上给一句"考虑以旧换新"，这种基于数据的小结论比多一个图表更让人记住。

核心界面：主列表按类别分组或按保修到期排序，每行显示设备名、类别和保修状态；详情页上方是产品图，中间是购买日期到保修截止日的一条横向时间轴；新增表单突出购买日期和保修时长，用"一年""两年""三年"的快捷按钮代替手选日期。

数据模型要点：主要用 `name`、`categoryName`、`quantity`、`expireDate`、`imageData`、`note`、`createdAt`、`updatedAt`，其中 `expireDate` 存保修截止日，`categoryName` 取"手机""电脑""配件"这类值。建议额外加 `purchaseDate`、`price`、`serialNumber`、`warrantyMonths`，`warrantyMonths` 配合 `purchaseDate` 可以自动算出 `expireDate`，这是省下用户输入成本的关键。

覆盖的进阶要求：进阶一（保修到期提醒、本地通知、临期排序）和进阶二（按类别统计总价值、可排序表格）都能做，进阶三加系统分享就能把一份设备清单发给别人。

工作量评估：二点五到三点五人天，按每天三小时算大约 7 到 10 天。大部分时间花在表单和自动计算保修期上，界面本身不复杂，是最容易提前完成的一个。

风险点：保修期只能手动录入，真实使用中用户坚持不下来，所以要默认给"一年"这个常见值并允许留空，不能把日期设成必填；分成两个日期字段后排序容易混乱，规则要写死在界面上，比如"主列表永远按保修截止日排"；价格和序列号属于敏感信息，演示时别用真实数据。

适合什么样的你：手边就有五件以上数码设备、愿意在信息结构上花心思的人。这个方向容易做完整，如果你的目标是稳过而不是惊艳，它是很好的保底选择。

2.4 方向三 收纳定位：东西放在哪

一句话定位：记录每件东西放在哪个柜子、哪个箱子、哪一层，找东西的时候搜一下就知道去哪拿。

用户与场景：适合搬过家、住宿舍、有换季收纳需求的人，也适合家里东西多、经常找不到充电线和备用配件的同学。典型场景是收纳完一批东西后集中录入，之后每次找东西只用搜索。

创意亮点：它把"分类"和"位置"明确区分开，这一点很多同类作品会做错。一个东西可以属于"电子配件"这个分类，同时放在"书桌第三个抽屉"，按分类浏览和按位置浏览是两条独立路径，而 `location` 字段正好支撑第二条路径。第二个亮点是位置自动补全：录过"书桌第三个抽屉"之后，再录同位置的东西时输入几个字就能选出来，避免"书桌抽屉三"和"书桌第三个抽屉"并存这种数据灾难。第三个亮点是"柜子视图"，把某个位置下的所有东西聚成一屏，柜子的实拍图作为背景，答辩时很容易被记住。

核心界面：主界面两个分段，一段按分类的列表，一段按位置的列表，位置页支持二级展开；搜索框常驻顶部，匹配名称、位置和标签；详情页突出"放在哪"这一行；新增表单提供常用位置的下拉建议和一键复制上一条的位置。

数据模型要点：核心是 `location`，其次用 `name`、`categoryName`、`quantity`、`tags`、`imageData`、`note`、`createdAt`、`updatedAt`。建议额外加 `locationId`、`spotPhoto`、`boxLabel` 三个可选字段，`locationId` 让位置变成可复用的独立实体，`spotPhoto` 存柜子或箱子的照片，`boxLabel` 存纸箱编号。`tags` 在这里可以当作"季节"或"归属人"用，比如标签"冬装""宿舍"。

覆盖的进阶要求：进阶二（标签、按位置统计、可排序表格）覆盖得最好，进阶四（缩略图缓存、导出清单）很契合，进阶一可以用"换季提醒"这种弱化形式挂上。

工作量评估：二到三人天，按每天三小时算大约 6 到 9 天。核心只有列表、筛选和表单，位置树的层级要自己控制，做两层就够。

风险点：录入成本比方向一更高，每件东西都要填位置，必须靠自动补全和"复制上一条"来压低操作次数；位置命名没有约束会迅速失控，最好内置一组常用位置；图片热区会让工作量翻倍，建议只对一两个柜子做，其余仍用普通列表。

适合什么样的你：手头有清晰的收纳体系、喜欢做工具型 App 的人。它是最快能做出可用版本的方向，界面朴素一点没关系，因为"搜一下就知道在哪"本身就是最好的卖点。

2.5 方向四 我的收藏图鉴

一句话定位：把手办、球鞋、徽章、书籍、唱片这类收藏做成一本可以翻的图鉴，看数量、看系列进度、看入手故事。

用户与场景：适合有明确收藏爱好的同学，比如手办、盲盒、模型、球星卡、实体书。典型场景是拿到新品后拍照入库，每隔一段时间翻一翻自己有多少、还差哪几个。

创意亮点：它把浏览体验做成核心，而不是把录入做成核心。封面以图片为主，名字压在图上，滚动起来像画册；`isFavorite` 在别的方向里是"收藏按钮"，在这个方向里变成"镇店之宝"标记，被标记的物品在首屏顶部横排成一条精选带。第二个亮点是系列进度：同一系列的物品用 `tags` 里统一的系列标签串起来，详情页显示"该系列 7 件，你有 4 件"，缺的那几件可以记成只有名称没有图片的条目，进度条立刻有了意义。第三个亮点是随机回顾，每天把一件三个月以上没看过的物品推到首页，靠 `createdAt` 和 `updatedAt` 就能算，不需要任何额外数据。

核心界面：首页是两列或三列的图片网格，顶部一条精选带，下方按系列分组；详情页大图加系列进度条和入手信息；新增表单只强调名称、分类、图片和系列标签，其余字段折叠；另有一个切换成列表的浏览模式。

数据模型要点：主要用 `imageData`、`name`、`categoryName`、`tags`、`isFavorite`、`createdAt`、`updatedAt`、`note`、`quantity`。建议额外加 `seriesName`、`acquiredAt`、`sourceNote`、`price` 和一个额外的 `imageDataList`，其中 `seriesName` 直接支撑进度统计，`acquiredAt` 把"入手时间"和"录入时间"分开，`sourceNote` 记在哪买的或谁送的。

覆盖的进阶要求：进阶二（标签、系列统计、图表看板）是主战场，进阶三的相机拍照和系统分享都很有用，进阶四的图片压缩与缓存几乎必做。

工作量评估：四到五人天，按每天三小时算大约 11 到 14 天。多出来的一天半基本都在图片上：压缩策略、网格滚动性能、大图查看的手势。

风险点：封面图占用存储最多，图片不压缩直接入库，五十条之后列表就卡，必须在保存时统一降采样；网格里图片宽高比不一会显得很乱，要固定单元格尺寸并用统一填充方式；系列进度依赖标签写法一致，如果用户手打成"EVA"和"eva"就统计不出来，需要对标签做归一化。

适合什么样的你：自己就有收藏品、手机里有现成照片、愿意把时间花在视觉呈现上的人。这是六个方向里最出效果的一个，但也是最容易被图片细节拖住的一个。

2.6 方向五 礼物与人情账本

一句话定位：记下送给谁什么、收到谁什么、什么场合、大概多少钱，避免下次重复送或者忘了回礼。

用户与场景：适合朋友多、经常参加生日和婚礼的同学，也适合需要维护客户或师兄师姐关系的人。典型场景是节日前后集中记录，以及赴约之前翻一下"上次他送过我什么"。

创意亮点：它给物品加了一个方向和一个对象，`quantity` 在这个方向里退居次要位置，取而代之的是"送给谁"和"什么时候"。第一个亮点是双向视图：同一条记录既能按人看，也能按时间看，按人看的时候一屏就是他这几年的往来。第二个亮点是重复礼物预警，送之前搜一下这个人的名字，如果半年前送过同类东西，界面上直接标出来，这个提醒是真能避免尴尬的。第三个亮点是金额只做区间不做精确数字，用几档带区间的选择代替输入框，既够用又不冒犯。

核心界面：首页两条入口，按人看和按时间看，按人看是联系人列表加每人一行最近记录；个人页是这个人收到和送出的时间线；记录页表单包含对象、方向、场合、日期、金额档位、图片和备注；另有一页"即将到来的日子"，把生日和纪念日提前列出来。

数据模型要点：主要用 `name`、`categoryName`、`expireDate`、`note`、`imageData`、`createdAt`、`updatedAt`、`tags`、`isFavorite`。这里 `expireDate` 存下一次需要回礼或生日提醒的日期，`tags` 存场合和关系，比如"婚礼""室友"。建议额外加 `personName`、`direction`、`occasion`、`priceRange`、`eventDate` 五个字段，其中 `direction` 用枚举表示送出和收到，是这个方向区别于其他方向的根本字段。

覆盖的进阶要求：进阶一（生日与回礼提醒、本地通知）天然契合，进阶二（按人、按场合、按年份统计）也好做，进阶三只用到图片和分享，进阶四用处不大。

工作量评估：三到四人天，按每天三小时算大约 8 到 11 天。多出的时间在于按人聚合的查询和两条浏览路径，如果只做一条路径，两天半能出可用版本。

风险点：记录里包含别人的姓名和金额，属于隐私数据，演示时要用化名，并且可以在设置里加一个应用内简易锁；按人聚合需要按字符串分组，姓名写法不一致就分不到一起，建议提供"从最近联系人里选"而不是纯手打；这个方向功能容易越做越多，一旦跨到记账和账目结算，两周肯定做不完，要守住"只记礼物"的边界。

适合什么样的你：社交场景多、觉得"记人情"这件事真实有用的人。这个方向的选题在班级里撞车概率很低，创意分好拿。

2.7 方向六 借还与共享

一句话定位：记录东西借给了谁、什么时候该还、自己借了别人什么，还能把一份清单分享给室友共同维护。

用户与场景：适合宿舍几个人共用工具、同学之间经常互相借书和借设备的场景，也适合社团管理公共物资。典型场景是借出时花十秒记一条，到期前收到提醒，归还时点一下标记。

创意亮点：它管理的不是静止的物品，而是物品的流动状态。第一个亮点是状态机：`note` 在这个方向里承载"借给谁、什么时候借的"，但更好的做法是把它显式化成"在库、已借出、已归还"三种状态，列表按状态分区，已借出的一眼能看到谁拿着。第二个亮点是提前自动填充：打开新增页时默认就是"借出、今天、未归还"，用户只需要填名字和物品，操作次数压到最少。第三个亮点是清单分享，没有后端也能做出半个共享：把当前借出列表生成为图片或文字，通过系统分享发到宿舍群，别人能看懂谁欠谁什么，这是很好的一个产品边界说明。

核心界面：主界面三个分段，在库、已借出、历史记录；借出卡片显示缩略图、借用人、借出天数和到期日；详情页有一个大的"已归还"按钮，点击后进入历史并记录归还时间；新增页预填方向和日期。

数据模型要点：主要用 `name`、`categoryName`、`quantity`、`imageData`、`note`、`expireDate`、`createdAt`、`updatedAt`、`tags`。`quantity` 在共用物资里很重要，因为同一个型号可能有几件；`expireDate` 存约定的归还日期。建议额外加 `borrowerName`、`lendDate`、`returnDate`、`status`、`contactHint` 五个字段，其中 `status` 用枚举，`borrowerName` 单独成列才能按人统计"谁借得最多"。

覆盖的进阶要求：进阶一的归还提醒和 Widget（显示逾期数量）非常契合，进阶二按借用人统计也可做，进阶三的分享是核心卖点之一。

工作量评估：三点五到五个人天，按每天三小时算大约 10 到 14 天。状态流转、历史记录和分享三块各占半天以上，比看上去要重。

风险点：没有后端就做不了真正的多用户共享，两个人各自手机上看到的是两份数据，这一点必须在文档里说清楚，把共享定义成"清单导出与分享"，不要承诺实时同步；借出和归还要经历状态变更，容易在两个页面之间出现状态不一致，规则要定成只有详情页能改状态；逾期判断同样要处理跨零点和"今天到期"的边界。

适合什么样的你：宿舍里确实有共用物品、愿意在状态设计上动脑、并且能接受"共享只做到分享这一步"的人。它是六个方向里创意上限最高的一个，也是唯一一个如果强行加多用户就可能翻车的方向。

2.8 六个方向横向对比

方向	创意度	工作量	最适合的情况
保质期管家	中	3 到 4 人天	想稳过且要覆盖进阶要求，手边有食品日用品
数码抽屉与保修簿	中低	2.5 到 3.5 人天	时间紧、想要保底，手边有数码设备
收纳定位：东西放在哪	中	2 到 3 人天	最快想出可用版本，有收纳和搬家需求
我的收藏图鉴	中高	4 到 5 人天	有现成藏品照片，愿意做视觉呈现
礼物与人情账本	中高	3 到 4 人天	社交场景多，想避开常见选题
借还与共享	高	3.5 到 5 人天	宿舍有共用物资，愿意做状态和分享设计

如果你的目标是先保证交得出来，排序建议是：方向三最容易做完，因为它的核心只有一个位置字段和两条浏览路径，两天就能跑通全流程；方向二紧随其后，字段多一点但界面简单。如果你的目标是在班里被记住，排序建议是：方向六最出彩，因为它有状态流转和分享这个真实的产品边界；方向四次之，图鉴式的浏览在答辩现场视觉优势明显。最稳的组合是这样：用方向三当主线保证两周内能交付，再把方向一的临期配色和本地通知作为进阶功能挂上去，这样既有一个不容易翻车的地基，又有一个能在演示最后三十秒亮出来的点。

第 3 章 推荐方案：保质期管家

3.1 为什么推荐这个方向

一句话结论：在"物品管理 App"这个题目里，把物品范围收窄到有保质期的食品、药品、化妆品，是投入产出比最高的做法。

理由有三条。第一，五条基础要求会被模型天然覆盖：物品就是食品和药品，分类天然存在（乳制品、生鲜、药品、护肤），图片有明确用途（拍包装上的日期），不需要硬凑"我给一本书拍了封面"这种勉强说辞。第二，进阶要求里最容易做出观感的是时间提醒，而时间提醒正好是保质期管家的核心卖点，不是事后贴上去的功能。第三，演示故事性强。评审看一个纯收纳 App 只能用"我存了 30 件东西"来证明价值，而你可以说"这瓶牛奶还有 3 天过期，App 昨天提醒过我"，这句话一秒就能讲清楚。

必须诚实说明两条局限。一是受众偏窄，不屯货、不做饭的同学对这个 App 没有刚需，所以设计上不要写成"只服务家庭主妇"，物品名称、分类都允许自由输入。二是首次录入成本高，用户要一件件拍照输入，前十分钟很容易放弃，所以新增流程要极短：名称输入框自动获得焦点，分类、数量、日期都给默认值，所有字段都可跳过。

> 结论：这个方向不是最酷的，但它让你在两周内"每条要求都能交出实物"，而不是"每条要求都差一点"。

3.2 MVP 功能地图

这里的 MVP 指"做完就刚好满足全部 5 条基础要求"的最小功能集，不含任何亮点模块。规模上控制在 6 个文件、约 800 行代码，读者一定做得完。

功能	对应哪条基础要求	属于哪个界面
列表展示所有物品，读取本地数据	第 1、3、5 条	物品列表
新增物品，填写名称与分类	第 1、2 条	新增编辑
点击列表行进入详情，查看全部字段	第 1、5 条	详情
详情页编辑后保存，删除需二次确认	第 1、5 条	详情
PhotosPicker 选图并随物品保存	第 2 条	新增编辑
按分类筛选、按名称搜索	第 4 条	列表

功能	对应哪条基础要求	属于哪个界面
App 重启后数据与图片都还在	第 3 条	全局

四条判断标准：能连续完成"新增到删除"的全流程而不报错；搜索和筛选能立刻改变列表结果；杀掉 App 重开数据不丢；每张图在列表里能显示为缩略图。

3.3 三个可选的亮点模块

模块一 到期提醒

做什么：给物品设置到期日，列表按剩余天数排序并用颜色区分状态，到期前 3 天和当天各推一条本地通知。

为什么加分：直接命中进阶要求第 1 条"时间提醒"，而且提醒是这个 App 存在的最强理由，演示时能实打实地弹出通知横幅。

实现要点：状态用计算属性算，不存数据库；通知用 UNUserNotificationCenter，日期用 Calendar 算天数差。

```
// 状态是从 expireDate 算出来的，不占数据库字段，改日期后自动更新
extension Item {
    var daysLeft: Int? {
        guard let expireDate else { return nil }
        return Calendar.current.dateComponents([.day], from: .now, to: expireDate).day
    }
}

// 有效期 3 天内才提醒；先请求授权，失败不阻塞主流程
let center = UNUserNotificationCenter.current()
try? await center.requestAuthorization(options: [.alert, .sound])
let content = UNMutableNotificationContent()
content.title = item.name
content.body = "还有 3 天到期"
let id = item.name + "-3d"
```

难度：三星。降级方案：通知做不完就只保留视觉层，列表里按临期标色排序——30 天内橙色、已过期红色、其余不标色，并在列表顶部加一条"3 件即将过期"的汇总文案。授权流程、权限拒绝处理全部砍掉，这一层没有任何系统交互，必定能做完。

模块二 存放位置

做什么：给物品记录存放位置（冰箱、药箱、厨房柜），位置用文本输入加历史值补全，列表支持按位置分组。

为什么加分：命中进阶要求第 2 条"更丰富的分类、标签、排序"，同时把"找得到"这件事做实，用户能回答"药箱里还有什么药快过期了"。

实现要点：位置直接存字符串，不需要新建模型；分组用 SwiftUI 的 Section 配合 Dictionary(grouping:)。

```
// 用位置做分组，位置为 nil 的物品统一归到"未指定"
let groups = Dictionary(grouping: items) { $0.location ?? "未指定" }
```

难度：两星。降级方案：只做详情页的一个位置输入框和列表按位置分组的 Section，不做历史值补全、不做位置管理界面。分组代码不到十行，是性价比最高的一个亮点。

模块三 分享与小组件

做什么：详情页用 ShareLink 把"名称加剩余天数"分享出去；主屏加一个小组件，显示最近要过期的三件物品。

为什么加分：命中进阶要求第 3 条"调用系统分享、Widget 等原生能力"，小组件摆在主屏上，演示时不用解锁 App 就能看到内容，观感分数很高。

实现要点：分享用 ShareLink 一行搞定；小组件要新建 Widget Extension，用 App Group 加共享容器读同一份 SwiftData 数据，时间线用 TimelineProvider。

```
// ShareLink 不需要自己写分享面板，系统会弹出标准 UI
ShareLink(item: "\(item.name) 还有 \(item.daysLeft ?? 0) 天到期") {
    Label("分享", systemImage: "square.and.arrow.up")
}
```

难度：分享一星，小组件四星。降级方案：小组件是三个模块里唯一"可能翻车"的，因为它涉及新 Target、App Group 和容器共享，配置错一步就读不到数据。时间紧就只做 ShareLink，把小组件降级为"App 内的到期统计卡片"，用 Charts 画一张未来 30 天到期数量的柱状图。统计卡片同样是进阶第 2、4 条的加分点，而且完全在主工程内，零配置风险。

3.4 界面地图

界面	作用	难度	是否必须做
物品列表	入口页，展示全部物品与缩略图	两星	必须
物品详情	查看完整字段，编辑与删除入口	一星	必须
新增编辑	同一个表单视图，新增与编辑复用	三星	必须
分类筛选	顶部横向分类条，点选后过滤列表	两星	必须

界面	作用	难度	是否必须做
搜索	搜索框按名称模糊匹配	两星	必须
统计	Charts 柱状图，看未来到期分布	三星	亮点模块才用
设置	通知开关、默认提醒天数、清空数据	两星	可选
空状态	无数据时显示图标、说明与"添加第一件"按钮	一星	必须

新增和编辑复用同一个视图，用可选的待编辑物品做参数，这是省时间的关键设计。空状态只需一个 VStack，但它是评审最容易注意到的细节，不要省。

3.5 数据模型与字段

字段沿用全文档统一的数据模型。下面只列字段，完整实现见数据模型章节。

```
@Model
final class Item {
    var name: String = ""
    var categoryName: String = "未分类"
    var quantity: Int = 1
    var expireDate: Date?              // 亮点一用
    var location: String?              // 亮点二用
    var note: String?
    var isFavorite: Bool = false
    var tags: [String] = []
    var createdAt: Date = .now
    var updatedAt: Date = .now
    @Attribute(.externalStorage) var imageData: Data?
}
```

字段	归属	说明
name、categoryName	MVP 必需	基础要求第 2 条点名的两个字段
quantity、createdAt	MVP 必需	数量参与显示，createdAt 用于默认排序
imageData	MVP 必需	externalStorage 自动把大图落到数据库外
expireDate	亮点一	到期提醒、临期标色、统计图都靠它
location	亮点二	位置分组
note、isFavorite、tags	亮点三	备注、收藏、标签筛选
updatedAt	可选	编辑时手动刷新，用于"最近修改"排序

为什么 expireDate 和 location 设计成可选值。这不是偷懒，是两个真实的产品判断。第一，一个 App 里必然存在没有保质期的物品，比如数据线、笔记本、洗发水；如果 expireDate 是非可选，列表里就会出现"数据线 1970 年过期"这类假数据，还会污染到期统计。第二，存放位置是次要信息，用户添加时通常只想拍一下包装，逼他填位置等于提高录入门槛；做成可选后，位置可以在详情页慢

慢补，补全率反而更高。技术上，可选值还能让派生状态的代码保持干净：没有日期就直接提前返回 .none，不需要靠魔法数字区分"未设置"和"已过期"。

3.6 技术选型清单

需求	用什么框架	关键 API	备注
数据持久化	SwiftData	ModelContainer、@Query、modelContext	用 .modelContainer 挂在 App 入口
图片选择	PhotosUI	PhotosPicker、loadTransferable	不要申请相册全量权限
图片存储	SwiftData	@Attribute(.externalStorage)	先压缩再存，否则数据库会膨胀
日期计算	Foundation	Calendar、DateComponents	不要用 TimeInterval 手动算天数
本地通知	UserNotifications	UNUserNotificationCenter、UNCalendarNotificationTrigger	需要真机或模拟器授权后验证
图表	Charts	Chart、BarMark	iOS 16 起可用，画到期分布
分享	SwiftUI	ShareLink	一行代码，无需第三方
小组件	WidgetKit	TimelineProvider、App Group	需要新建 Target，最容易出错
状态管理	Observation	@Observable、@State	表单与筛选条件用它管理即可

> 提示：图片压缩放在选图之后、写库之前，用 UIKit 的 jpegData(compressionQuality: 0.7) 把图压到几百 KB 再赋值给 imageData。这一步不做，几十张图之后列表滚动就会卡。

3.7 这个方案能拿到什么评价

对照题目逐条看。基础要求第 1 条，列表、详情、新增编辑三个界面形成完整闭环，删除带确认，属于达标偏上。第 2 条，名称、分类、数量齐全，图片是拍包装这个真实用途而不是装饰。第 3 条，SwiftData 加 externalStorage，重启后文字和图片都在。第 4 条，分类筛选与搜索两个入口都做了。第 5 条，App 有明确的日常使用场景，不是只能演示的花架子。

进阶要求第 1 条由到期提醒拿下，第 2 条由位置分组和统计图拿下，第 3 条由 ShareLink 和小组件拿下，第 4 条由图片压缩加异步加载拿下。四条全覆盖，且每条都有可演示的实物。

最容易做出"超出预期"观感的地方有四处。一是列表行的临期标色与排序，让列表一眼就有信息密度。二是通知真的弹出来那一刻，演示的说服力远高于任何 UI 细节。三是空状态，绝大多数同学会忘记做。四是小组件摆在主屏上，评审在你打开 App 之前就已经看到了你的作品。四处里前三处成本都很低，建议全做，第四处按时间决定。

第 4 章 让作品显得有创意的 18 个低成本点子

下面的点子都按 iOS 17+ 的 SwiftUI 写法给出，数据模型沿用全书的 Item。每个点子只点出关键 API，不展开成完整代码；难度星级从半天的活到一次重构。

4.1 交互细节类

1. **左滑删除加撤销**。给列表行加 `.swipeActions(edge: .trailing)`，删除按钮写 `role: .destructive`；真正的删除延后三秒执行，屏幕底部弹出一条带"撤销"的提示。删除是不可逆操作，先给用户一次后悔机会，评委会把它读成产品意识。撤销条用 `.overlay(alignment: .bottom)` 加 `.transition(.move)` 就够了。工作量约半天。（★★☆）
2. **长按多选批量操作**。列表进入编辑态后用 `@State` 存一个 `Set`，元素是 `PersistentIdentifier`，顶部工具条出现"批量改分类""批量删除"。单条编辑要重复点很多次，批量入口把整理动作压缩成两步，也顺便演示了集合类型和状态管理。用 `.toolbar` 切换按钮和 `.confirmationDialog` 收尾。工作量约半天。（★★☆）
3. **列表滑到顶部的排序切换**。在 `.safeAreaInset(edge: .top)` 上放一个胶囊形的排序条，点一下在按创建时间和按到期时间之间切换，列表顶部同时淡入排序说明。它把筛选状态固定在视线范围内，比藏进菜单更容易被看见，也能配合 `.animation` 做平滑重排。排序键只用 `createdAt` 和 `expireDate`。工作量约两小时。（★★☆）
4. **搜索历史加热门标签**。搜索框聚焦且没有输入时，下方列出最近五次搜索词和全部标签的去重结果，点一下直接填入查询。空搜索页最容易被做成一片白，放上历史与标签就变成了导航入口。历史用 `@AppStorage` 存成字符串数组，标签从物品里现算。工作量约两小时。（★★☆）
5. **空状态的两种写法**。用 `ContentUnavailableView` 区分"一件物品都还没有"和"筛选后没有结果"：前者给一个"添加第一件物品"的按钮，后者给"清除筛选条件"。很多作品从头到尾只有一个列表，筛选到空时白屏，用户会以为程序崩了。关键字是 `.searchable` 的查询绑定和 `ContentUnavailableView.search`。工作量约一小时。（★☆☆）

4.2 视觉呈现类

6. **加载骨架屏**。数据还在读的时候，用几行占位卡片配上 `.redacted(reason: .placeholder)`，再叠一层横向扫过的高光做微光效果。首屏不再抖动，观感接近成熟商业 App，实现却只是几行视图代码。关键是骨架的高度要和真实卡片一致，否则切换时会跳一下。工作量约两小时。（★☆☆）

7. **临期看板用卡片而不是列表**。把三十天内到期的物品排成一列大卡片，每张显示缩略图、剩余天数和状态色，超过三十天的收进下面的分组列表。卡片天然承载图片和进度信息，一眼就能看出紧迫程度，比十条一模一样的行更像"看板"。用 `LazyVStack` 加 `.background(.regularMaterial)` 和圆角即可。工作量约半天。（★★☆）

8. **详情页头图加信息分组**。顶部放一张十六比九的图片，上面压一层从透明到深色的渐变让白字可读；下面用三个带小标题的区块分别放基本信息、时间与位置、备注，区块之间用间距而不是分隔线。这是把"表单回显"变成"内容页"的最短路径。图片用 `.resizable().scaledToFill().frame().clipped()`。工作量约两小时。（★☆☆）

9. **统计页用 Charts 看分类占比**。引入 Charts 框架，用 `SectorMark` 画分类占比的环形图，旁边配一条按时长的 `BarMark` 表示每月新增数量，点扇区还能把对应分类筛进列表。图表是进阶要求里最容易做出效果的一项，代码量却只是两个可组合的视图。数据先聚合成 `[(String, Int)]` 再喂给 `Chart`。工作量约半天。（★★☆）

10. **分享卡片用 ImageRenderer 导出图片**。把一件物品的图片、名称、分类和到期日排成一张固定尺寸的卡片视图，用 `ImageRenderer` 渲染成 `UIImage`，再用 `ShareLink` 分享出去。截图分享比分享纯文本更有说服力，也让评委看到你理解了 SwiftUI 视图的离屏渲染。注意在主线程上渲染，并把 `scale` 设成 3。工作量约半天。（★★☆）

4.3 原生能力类

11. **Widget 显示临期数量**。用 WidgetKit 做一个小组件，显示"三十天内到期：3 件"，点一下通过 `widgetURL` 深链回 App 的临期列表。这是进阶要求里明确点名的原生能力，能同时展示 App Group 共享容器和 Timeline 刷新两条知识点。数据用 `.containerURL` 指向共享目录，时间线按小时刷新。工作量约一天，是本清单里最贵的一项。（★★★）

12. **App 图标与启动页视觉统一**。图标用单一底色加一个白色物品符号，不要塞文字；启动页用同一底色加居中标志，和首屏的配色逐项对齐。启动页到首屏的过渡如果不闪白，整个作品会显得收敛、有设计稿。图标在 Assets 里按单尺寸交付即可，启动页用 Info.plist 的启动屏幕配置。工作量约两小时。（★☆☆）

13. **深色模式的颜色一致性**。所有自定义颜色放进 Asset Catalog 的颜色集，同时配好 Any 和 Dark 两个外观；代码里只用 `.primary`、`.secondary` 和 `Color(.systemGroupedBackground)` 这类语义色。写死 `Color.white` 当背景在深色模式下会直接失效，而评委几乎一定会切换外观看一眼。改完在模拟器的外观切换里逐页走一遍。工作量约两小时。（★☆☆）

14. **键盘工具栏与表单体验**。给名称、数量等输入框加 `.toolbar { ToolbarItemGroup(placement: .keyboard) }`，放"上一项""下一项""完成"，数量用 `.keyboardType(.numberPad)` 并把焦点交给 `@FocusState` 管理。表单能连续填写而不用反复收键盘，试用时的手感差别非常明显。同时给保存按钮加 `.disabled(name.isEmpty)`。工作量约两小时。（★☆☆）

15. **无障碍与动态字体**。给只有图标的按钮补 `.accessibilityLabel`，给图片补描述；字号统一用 `.font(.headline)` 这类文本样式或 `.dynamicTypeSize` 可控的写法，不要写死 `.system(size: 13)`。把系统字号调大后界面不截断，是无障碍的第一道门槛，也是评委翻设置时容易发现的问题。用模拟器的环境覆盖快速检查两级大字号。工作量约两小时。（★☆☆）

4.4 工程与演示类

16. **README 里配 GIF 演示**。在 README 顶部放两张动图，一张走"新增带图片的物品"，一张走"搜索并查看临期提醒"，正文用表格列出功能与文件对应关系。评审通常先看仓库首页，一张动图抵得上一段功能列表，也省掉了对方动手编译的成本。动图用系统的录屏转 GIF，控制在两兆以内。工作量约两小时。（★☆☆）

17. **三分钟演示视频的脚本**。开录之前先写分镜表：每段时间、你的操作、你要说的话、想展示的亮点，写满一张 A4 再开录。视频脚本本身不写代码，却决定了答辩时能不能把亮点讲完，也能提前暴露"某个功能其实演示不出来"的问题。录屏用模拟器的录制功能，导出后用系统剪辑裁掉等待。工作量约半天。（★☆☆）

18. **把"我放在哪了"做成搜索建议**。搜索时除了匹配名称，也匹配 `location` 和 `tags`，命中位置时结果行右侧显示位置徽标，并把"在抽屉里"这类查询映射成按位置分组的结果。这是本题目真正的使用场景，用户记不住名字、只记得大概放在哪。实现只需在过滤闭包里多判断两个字段，并把匹配字段回显出来。工作量约两小时。（★★☆）

第 5 章 14 天实施排期

5.1 三个里程碑

排期不是为了把日程填满，是为了让你在三个时间点知道自己有没有跑偏。每个里程碑都有可以动手验证的通过标准，不靠感觉判断。

里程碑一，第 7 天基础要求全通。通过标准：新建一个空工程后，能连续完成"新增一件带图的物品、在列表看到它、进入详情、编辑名称、删除它"这一整条流程且不报错；杀掉 App 重新打开，刚才的数据与图片仍在；分类筛选与搜索都能立刻改变列表；五个界面之间导航正确，没有死路。此时不要求任何亮点功能，界面丑一点没关系。

里程碑二，第 11 天亮点模块可用。通过标准：给一件物品设置 3 天后的到期日，列表按剩余天数排序且该行显示为橙色，模拟器能在设定时间弹出本地通知；按存放位置分组显示正常；ShareLink 能拉起系统分享面板并分享出正确文案。这里允许小组件未完成，但到期提醒必须能演示。

里程碑三，第 14 天可演示可提交。通过标准：一份三分钟的演示视频，从空数据开始，完整走完新增、筛选、查看、提醒、删除；README 写清项目简介、运行方式与已实现功能清单；深色模式下界面没有白底黑字；编译无错误、无警告；工程能在老师指定的模拟器机型上一键跑起来。

> 提示：里程碑一拖到第 9 天才完成，就立刻执行 5.4 的压缩方案，不要硬撑着往后赶。

5.2 逐日计划

每天按 2 到 3 小时估算。如果每天只有 1 小时，把第 7 天之后的顺序整体后移，但 5.4 的取舍方案始终有效。

天	目标	做什么	当天产出
1	工程能跑	建工程、选 SwiftData 模板、建 Item 模型、挂 modelContainer，用 #Preview 造三条假数据	工程可运行，预览里能看到假数据
2	列表能显示	写物品列表，用 @Query 读数据，NavigationStack 包起来，行内显示名称、分类、数量	首页能列出全部物品，点击不崩
3	详情能看	做物品详情页，展示全部字段，加删除按钮与二次确认，跑通列表到详情的导航	增删查导航闭环，可删数据

天	目标	做什么	当天产出
4	新增能填	做新增编辑表单，名称与分类用 TextField、数量用 Stepper，保存写回 modelContext	新增一件物品后立刻出现在列表
5	编辑复用	让同一个表单兼顾编辑，传可选物品做参数，补上 updatedAt 刷新与取消逻辑	编辑后列表同步更新
6	图片能用	PhotosPicker 选图、loadTransferable 取 Data、压缩到几百 KB、赋值 imageData，列表显示 56 点缩略图	带图物品在列表显示缩略图
7	基础全通	补分类筛选与搜索，然后做一次完整回归：五条基础要求逐条走一遍并记录问题	基础要求全通，问题清单一份
8	日期进模型	表单加到期日 DatePicker，写 daysLeft 与 expiryState 计算属性，列表按到期排序	物品能设置日期并正确排序
9	提醒能弹	写到期状态标色，封装通知调度，请求授权并在启动与回到前台时重排通知	临期行变色，通知能弹出
10	位置能分组	详情与表单加位置输入，列表用 Dictionary 分组配 Section 展示	列表可按位置分组浏览
11	分享与统计	详情页加 ShareLink，统计页用 Chart 与 BarMark 画未来 30 天到期分布，时间够就试 Widget	分享面板可用，统计图能出图
12	视觉与空状态	统一间距圆角与语义色，补空状态页，处理无图与长文本的占位	空数据界面不空白，视觉统一
13	稳与快	处理授权拒绝、日期为空、删除最后一行的边界，检查深色模式，用一百条数据压测列表	无崩溃、无编译警告，滚动流畅
14	收尾提交	录三分钟演示视频，写 README，清理调试代码与假数据，确认一键运行	视频、README、可提交工程

第 6 天的图片流程是整份排期里技术风险最高的一天，压缩与写库的顺序不要颠倒。第 9 天的通知建议先在模拟器上把时间设成几十秒后验证一次，确认无误再改成真实天数。

5.3 每天收工前的自检

收工前固定花五分钟回答下面几个问题，答不上来的就是明天的第一件事。

- 今天做的功能，杀掉 App 重新打开还在吗？数据是不是真的写进了 SwiftData，而不是只改了内存里的数组？
- 换一个模拟器机型，从 iPhone SE 尺寸到 Pro Max 尺寸，界面有没有被截断或者挤成一团？
- 切到深色模式看一眼，有没有自己写死的白色背景或黑色文字？

- 今天的代码编译有没有警告？把残留的未使用变量和强制解包清掉，别留到第 13 天一起处理。
- 空数据、无图片、日期为空的三种情况，界面会不会崩或者显示成一片空白？
- 今天新增的代码有没有复制粘贴忘了改的变量名？尤其是新增编辑复用的那个表单。

5.4 时间只剩一半怎么办

如果第 7 天结束时你只完成了列表和新增，剩下的时间只够做一半，就走 7 天版压缩方案。顺序是：第 1 天建模型加列表，第 2 天详情与新增编辑，第 3 天图片，第 4 天分类筛选与搜索并做一次基础回归，第 5 天到期日与临期标色，第 6 天视觉打磨与空状态，第 7 天修边界、录视频、写 README。

压缩方案里砍掉的是：小组件、统计页、位置分组、通知的实际推送、设置页、标签与收藏。保留的是：五条基础要求、图片、临期标色与按到期排序、分享按钮、空状态。取舍逻辑是"保留看得见的、砍掉要配置的"：标色和排序是纯代码，二十分钟能写完而且一眼能看出效果；小组件要新 Target 和 App Group，配置踩坑可能吞掉一整天。分享按钮一行代码就能留，属于白捡的加分项，不要砍。

如果连 7 天都没有，就只做 MVP：列表、详情、新增编辑、图片、分类筛选加搜索、本地持久化，到期日字段保留在模型里但暂不做提醒。演示时用一句话说明"时间提醒已在数据结构中预留，下一步接入本地通知"，比做一个半成品的通知更能得分。最不该砍的是空状态和深色模式检查，这两项成本极低，却直接影响评审对"完成度"的判断。

第 6 章 这个题目上最容易翻车的 12 个地方

这一章按出现频率排序：前面的坑几乎每个作品都会踩，后面的坑决定你的演示能不能撑到最后。

6.1 数据与图片

坑 1：把图片直接存进普通属性

现象：添加十几张手机拍的照片后，App 启动变慢，模拟器里的数据库文件涨到几百兆，导出数据时卡住。

原因：普通 `Data` 属性会被整块写进数据库行，读取列表时整张图都要进内存；手机原图一张就有三到五兆。

正确做法：属性声明加上外部存储标记 `@Attribute(.externalStorage) var imageData: Data?`，并且写入之前先压到长边 1024、质量 0.7，例如 `image.jpegData(compressionQuality: 0.7)`。

坑 2：以为 PhotosPicker 选出来的是 UIImage

现象：`.photosPicker` 的回调里拿到一个 `PhotosPickerItem`，直接当成图片用会编译不过或拿到空数据，列表里始终没有图。

原因：`PhotosPickerItem` 遵循 `Transferable`，它只是相册里那条资源的引用，图片数据要异步取回来。

正确做法：先取出 Data 再交给模型，大致是 `if let data = try? await item.loadTransferable(type: Data.self)`，然后回到主线程更新 `imageData`。

坑 3：把 @Model 写成结构体

现象：定义 `struct Item` 并加上 `@Model`，编译直接报错，提示这个宏只能作用于类。

原因：SwiftData 靠引用语义做对象图管理、变更追踪和关系维护，值类型语义与它不兼容。

正确做法：写成 `@Model final class Item`，属性全部给默认值，例如 `var name: String = ""`，这样以后加字段也能顺利迁移。

坑 4：在 #Predicate 里调用字符串方法

现象：想按名字模糊搜索，写 `#Predicate { $0.name.localizedStandardContains(kw) }`，编译报错或运行到取数据时才崩溃。

原因：`#Predicate` 会被宏翻译成可在存储层求值的表达式，只支持一部分操作符，任意 Foundation 方法不在其中；闭包外部变量也必须先赋给局部常量。

正确做法：条件简单时用 `#Predicate { $0.name == keyword }`；要做不区分大小写的包含匹配，就先把查询结果取出来，再在内存里用 `items.filter { $0.name.localizedCaseInsensitiveContains(keyword) }` 过滤。

坑 5：用时间间隔除以 86400 算天数

现象：下午三点添加的物品，第二天早上看到剩余天数少了一天；跨夏令时或时区变化后数字更离谱。

原因：`Date` 之间相差的是绝对秒数，除以 86400 得到的是绝对秒数换算出的整天数，不是按日历算的自然日差，也忽略了日历和时区的规则。

正确做法：统一走日历，`Calendar.current.dateComponents([.day], from: .now, to: expireDate).day`，再按结果判断是否小于零、是否在三十天以内。

坑 6：把日期存成字符串

现象：模型里加一个 `expireText: String` 存"2026-03-01"，排序时按字典序排，出现"2026-10-01"排在"2026-03-01"前面。

原因：字符串的字典序和时间的先后顺序没有关系，写成固定格式也不等于可以比较，而且没法直接用 `DatePicker` 编辑。

正确做法：模型里保留 `var expireDate: Date?`，输入用 `DatePicker`，展示时才用 `date.formatted(date: .abbreviated, time: .omitted)` 转成文字。

6.2 交互与状态

坑 7：用中文名排序

现象：按名称排序后，列表顺序看起来是随机的，"阿莫西林"跑到"纸巾"后面，用户和评委都会觉得排序坏了。

原因：Swift 的字符串比较基于 Unicode 标量，字符串会逐码点比较，结果既不是拼音序也不是常用字顺序，与直觉不符。

正确做法：排序键只用 `createdAt` 或 `expireDate`，需要分组时用 `Dictionary(grouping:)` 按分类名分组，组内再按时间排。

坑 8：ForEach 用了不稳定的 id

现象：删除中间某一行后，剩下的行缩略图错位，或者界面出现重复 id 的运行时警告。

原因：`ForEach` 靠 id 判断哪些行是同一个视图；用数组下标 `id: \.self` 或 `id: \.offset` 做 id 时，删除会让所有下标整体前移，SwiftUI 就把旧视图复用到了新数据上。

正确做法：`@Model` 的 `Item` 天然有稳定的 `persistentModelID`，直接 `ForEach(items) { item in ... }` 即可；自己写的值类型结构体要遵循 `Identifiable` 并用 `UUID` 当 id。

坑 9：用 .onDelete 时索引和 @Query 结果对不上

现象：列表删掉的是别的行，或者直接崩溃在数组越界。

原因：`.onDelete` 回调给的是它在当前渲染数组里的下标，如果你先做了筛选、分组或排序，`items[offsets]` 拿到的已经不是同一个数组。

正确做法：让传给 `ForEach` 的数据和 `onDelete` 关的是同一个数组，`items.remove(atOffsets: offsets)`；更稳的写法是绕开下标，直接持有模型对象再 `modelContext.delete(item)`。

坑 10：大列表卡顿与缩略图反复解码

现象：放进两百条物品后列表滑动掉帧，缩略图一边滑一边重新闪出来。

原因：每行都做了一次 `UIImage(data:)` 和缩放，这发生在滚动过程中，等于每一帧都在解码大图。

正确做法：行高固定的长列表用 `List`（它自带复用）；网格或卡片流用 `LazyVGrid` 而不是 `VStack`；缩略图按显示尺寸缩放后再缓存，别直接塞原图，也避免在 `body` 里做解码。

坑 11：深色模式下写死白色背景

现象：浅色模式看着正常，一开深色模式详情页变成一大片白底白字，卡片和文字糊在一起。

原因：`Color.white` 和 `.background(.white)` 不随外观变化，而上方的文字用了 `.primary`，于是自动变成了白色。

正确做法：改用语义色，背景用 `Color(.systemGroupedBackground)`、`Color(.secondarySystemGroupedBackground)`，文字用 `.primary`、`.secondary`；确实要自定义颜色就放进 Asset Catalog 的颜色集并配好 Any 与 Dark 两套值。

6.3 时间与提醒

坑 12：想在提醒时再要通知授权

现象：给物品设了到期提醒，到点什么都不弹；或者你把授权弹窗留到演示途中才触发，用户点了不允许，之后永远收不到提醒。

原因：`.alert`、`.sound`、`.badge` 这些通知能力必须先拿到授权才能生效，而 SwiftUI 的 `NotificationCenter` 与 `UNUserNotificationCenter` 是两码事。

正确做法：在首次进入提醒设置页、用户第一次开启提醒时调用 `try await UNUserNotificationCenter.current().requestAuthorization(options: [.alert, .sound, .badge])`，拿到 false 就在界面上写明提醒已在系统设置里关闭，并把临期信息用红色徽标在 App 内展示，做到拒绝授权也不影响主流程。

6.4 演示与提交

正式提交是在三分钟演示之后做的最后一件事，这里的错通常不是代码错，而是准备不足。仓库里要放好 `.gitignore`，忽略 `xcuserdata/`、`.DS_Store`、`DerivedData/`；容器用 `ModelContainer(for: Item.self)` 让系统决定落盘位置，不要写死某台机器上的绝对路径；README 里补上运行环境、功能清单表格、三到五张关键界面截图和一段录屏动图。这四件事各自只要十分钟，却是评分人接触你作品的第一步，验收自检表里的 7.3 与 7.5 就是照这个顺序列的。

第 7 章 对照题目要求的验收自检表

这一章是可以逐条打勾的清单。建议在答辩前一天完整走一遍，每条都动手点，不要凭印象确认。

7.1 基础要求自检

要求	自检问题	达标标准
添加、查看、编辑、删除	添加一条带图片的物品，杀掉 App 重开，它还在列表里吗；点进详情改掉名称后返回，列表那一行跟着变了吗；对它左滑删除，它会消失并留下一次撤销机会吗	四个动作各点通一次，且改完不需要手动刷新界面
名称、分类等基本信息与图片	新增页面里只填名称能保存吗；分类、数量、存放位置、备注、到期日是否都有输入入口；相册选一张图后列表行和详情页都显示出来了吗	除名称外都可留空，保存后各字段与输入一致
数据本地保存	添加三条记录后从后台完全划掉 App 再重开，列表还是三条吗；连开飞行模式重开一次还正常吗	数据与图片都留存，界面没有请求网络才显示的内容
分类、搜索或筛选	把物品分到两个以上分类，点分类能筛出对应条目吗；搜"抽"能出"抽屉里的电池"吗；搜不到时界面提示清楚吗	三种入口至少实现两种，且空结果有空状态提示
完整可用的操作流程	从空库开始，添加、查看、编辑、搜索、删除连着走一遍会卡在哪一步吗；有没有哪个按钮点下去没有反应	主流程没有死路，每个页面都能返回上一级

7.2 进阶要求自检

要求	自检问题	达标标准
到期、保修或其他时间提醒	把一件物品的到期日设在三天后，重新打开 App 时它在临期区域里吗；授权弹窗第一次出现时你点了不允许，App 还能正常用吗	列表或看板按自然日算出剩余天数，通知授权被拒绝时有降级说明

要求	自检问题	达标标准
更丰富的分类、标签、排序或统计	分类是固定几个还是能自己加；一条物品能挂两个以上标签吗；排序在按加入时间和按到期时间之间切换吗；统计页有图表吗	分类、标签、排序、统计中至少实现三项，统计页至少有一个 Charts 图表
相机、系统分享、Widget 等原生能力	Widget 加到桌面后能显示临期数量吗；详情页的分享按钮能导出图片并调起系统面板吗	至少实现一项，并在演示里当场触发一次
大量物品、图片存储或数据管理的优化	用脚本或手动添加一百条物品后列表顺滑吗；图片是压缩后存的吗；数据库文件有多大	有明确的压缩或懒加载处理，翻到列表底部不掉帧不闪图

7.3 代码与工程自检

- 编译输出里没有黄色警告，特别是未使用变量和隐式强解包这两类。
- 全局搜索 `print(` 并清理，调试信息用 `Logger` 或临时断点代替。
- 没有写死的绝对路径，文件与容器位置都通过 API 获取。
- `.gitignore` 覆盖 `xcuserdata/`、`.DS_Store`、`DerivedData/`，仓库里没有构建产物。
- Assets 里只有被真正用到的图片，声明的 App 图标有实际内容，仓库里没有孤立资源。
- 模型的每个新增属性都有默认值，能直接通过 SwiftData 的轻量迁移。
- 分类名、状态色这类常量集中放在一个文件里，不在多个视图里重复出现。
- 提交前用一套干净的演示数据替换掉开发期的测试数据，删掉名字像"aaa""测试 12"的条目。
- 列表在一百条数据下滚动跟手，缩略图不闪烁，点击响应没有可感知的延迟。
- README 顶部有功能清单、截图和运行说明，写清 Xcode 版本、模拟器型号和首次启动为空库。

7.4 三分钟演示脚本

时间段	你做什么操作	你讲什么	想展示的亮点
0:00 - 0:20	展示首屏，如果有桌面小组件就让它一起入镜	一句话定位：这是一个帮你在需要时找到东西、在过期前提醒你的本地物品管理 App	一句话说清定位，桌面组件顺带证明原生能力

时间段	你做什么操作	你讲什么	想展示的亮点
0:20 - 0:50	点加号，填名称与分类，选一张相册图片，设置三十天后的到期日，保存	强调数据只存本地、不上传，图片会自动压缩后保存	添加流程顺畅，表单支持键盘工具栏
0:50 - 1:30	回到列表，进详情改分类，返回后左滑删除并点撤销，再用搜索找一个只记得位置的物品	说明编辑后列表即时更新，删除有一层保护	左滑撤销与按位置搜索，属于细节加分项
1:30 - 2:00	打开临期看板，指出卡片上的剩余天数与状态色，点开一张卡片看详情	到期天数按自然日算，三十天内自动进入看板	卡片式看板与状态色，比纯列表更有信息层次
2:00 - 2:25	切到统计页，点环形图里的一个扇区，跳回按该分类筛选的列表	分类占比可以点开下钻，数量靠 Charts 直接算	Charts 图表与跨页联动
2:25 - 2:45	打开详情页，点分享，系统分享面板弹出刚渲染好的卡片图	分享卡片由 ImageRenderer 导出，图片带名称与到期信息	系统分享与离屏渲染
2:45 - 3:00	回到首屏，说明数据保存在本地数据库，并打开深色模式	收尾：全部数据在本机数据库，图片单独存放，深浅色都适配过	深色模式一致性，收尾干净

7.5 提交前最后一遍

1. 清理调试代码：删掉残留的 `print`、临时代码和测试数据。

2. 重新拉一遍代码：从远端克隆到另一个目录，用同一个 Xcode 版本重新编译一次。

3. 全新环境跑一遍：删掉模拟器上的 App 重装，从空库开始走完 7.1 的五条基础要求。

4. 换设备尺寸复核界面：在小屏手机上确认长名称、大字号没有截断或重叠。

5. 检查截图与动图：README 里的截图来自当前版本，不是旧界面。

6. 确认仓库有 README 和 `.gitignore`，并且没有把构建产物提交上去。

7. 通读 README：克隆地址、运行环境、功能清单、已知限制都写清楚。

8. 最后按 7.4 的脚本完整演示一遍并计时，确认能压在三分钟以内。